**Experiment No: 08**

**Title:** Implement Binary search tree with operations Create, search, and recursive traversal.

**Aim:** Implementation of Inorder tree traversal using C**.**

Mapped to
**Course Objective:**
1.To solve problems using data structures such as trees
Mapped to
**Course Outcome:** After performing this experiment, student should be able to understand how the concept of tree traversal technique application.This also helps students to realize how in different ways operations on tree traversal can be implementedbe implemented.

**Theory:**

Linear data structures such as stack, array, queue, etc., only have one way to traverse the data. But in hierarchical data structures such as **tree,** there are multiple ways to traverse the data. Here we will discuss another way to traverse the tree data structure, i.e., inorder traversal.

- Inorder traversal using Recursion
- Inorder traversal using an Iterative method

An inorder traversal technique follows the **Left Root Right** policy. Here, Left Root Right means that the left subtree of the root node is traversed first, then the root node, and then the right subtree of the root node is traversed. Here, inorder name itself suggests that the root node comes in between the left and the right subtrees.

We will discuss the inorder traversal using both recursive and iterative techniques. Let's first start with inorder traversal using recursion.

Code:

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   int data;
```

```
    struct node *left,*right;
};

struct node* create(int x){
    struct node *newnode;
    newnode=(struct node*)malloc(sizeof(struct node));
    newnode->data=x;
    newnode->left=NULL;
    newnode->right=NULL;
    return newnode;
}

struct node* insert(struct node *root,int x){
    if(root==NULL)
        return create(x);

    if(x<root->data)
        root->left=insert(root->left,x);
    else
        root->right=insert(root->right,x);

    return root;
}

void search(struct node *root,int x){
    if(root==NULL)
        printf("Element not found");
    else if(root->data==x)
        printf("Element found");
    else if(x<root->data)
        search(root->left,x);
    else
        search(root->right,x);
}

void inorder(struct node *root){
    if(root!=NULL){
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
    }
}

int main(){
    struct node *root=NULL;
    int n,x,i,key;

    printf("Enter number of nodes: ");
```

```
    scanf("%d",&n);

    for(i=0;i<n;i++){
        scanf("%d",&x);
        root=insert(root,x);
    }

    printf("Inorder Traversal: ");
    inorder(root);

    printf("\nEnter element to search: ");
    scanf("%d",&key);

    search(root,key);

    return 0;
}
```

Output:

```
Enter number of nodes: 5
50 30 70 20 40
Inorder Traversal: 20 30 40 50 70
Enter element to search: 40
Element found
```